Agent Seer Meets Schema-Blindness: Spec-Driven Evaluation of Generative-Media MCP Agents

G. Hussain Chinoy
ghchinoy@gmail.com
Independent Researcher

Abstract

Autonomous AI agents that call external tools need realistic test scenarios. Building them by hand does not scale, and live execution is slow and costly in generative-media domains. Agent Seer (arXiv:2608.26133) generates these scenarios from tool specifications alone. We reproduce its pipeline across three production media servers: Google Veo, Nanobanana, and Lyria. We find a critical gap we call Schema-Blindness. Standard JSON schemas hide runtime model constraints, so an LLM judge scores broken calls as perfect (TC = 1.000). Injecting a machine-readable capability matrix fixes this. It restores discrimination gaps of ≥ 0.191 and widens the image-generation margin by 36.1%. We also propose a three-layer taxonomy that separates cheap orchestration checks from costly media-quality evaluation.

1 Introduction

The deployment of large language model (LLM)-based agents capable of autonomously orchestrating external tools is increasingly central to modern enterprise automation. In creative and generative-media workflows—such as automated video campaign synthesis, multi-model image generation, and dynamic soundtrack composition—agents must coordinate complex, multi-parameter tool calls across evolving interfaces. Validating tool-use competence in these domains has become a critical gate for reliable deployment.

However, evaluating autonomous agents against modern interface protocols, such as Anthropic's Model Context Protocol (MCP), encounters three severe bottlenecks:

1. **The Curation Bottleneck:** Handcrafting multi-turn user prompts, parameter arguments, and gold-standard oracle tool sequences requires deep domain expertise and prohibitive human engineering effort.
2. **The Static Benchmark Problem:** Hardcoded evaluation benchmarks rapidly rot as API schemas, parameter names, model enums, and runtime constraints iterate across releases.
3. **The Multi-Turn Evaluation Gap:** Conversational agents require evaluation across multi-turn interaction sequences where downstream turns react dynamically to intermediate tool outputs without incurring expensive live API execution.

In generative-media domains (e.g., video diffusion, neural image synthesis, audio composition), these challenges are amplified by long execution latencies (30–120 seconds per generation), high cloud inference costs, and stochastic output variance. Running live execution loops simply to test whether an agent can correctly structure a tool call is economically and architecturally unviable.

Recently, **Agent Seer** (Karumuri et al., arXiv:2608.26133) established that **tool specifications—consisting of function names, natural-language documentation, and typed JSON schemas—encode sufficient latent semantic structure to autonomously synthesize end-to-end evaluation suites without manual curation or live tool execution.**

In this work, we deconstruct, empirically reproduce, and substantially extend the Agent Seer methodology across three production-grade generative-media MCP server suites: Google Veo (`mcp-veo-go`), Gemini Image / Nanobanana (`mcp-nanobanana-go`), and Google Lyria (`mcp-lyria-go`).

1.1 The Primary Contributions

The primary contributions of this work are:

1. **Faithful Empirical Reproduction:** We provide the first comprehensive, multi-server reproduction of the complete Agent Seer pipeline applied to generative-media MCP interfaces, achieving 100% tool coverage across single-turn and multi-turn workflows while verifying mathematical scoring assertions and decomposed rubric mechanics.
2. **Identification of the Schema-Blindness Vulnerability:** We uncover a critical, negative finding: raw JSON tool schemas (`tools/list`) omit runtime backend model compatibility constraints (e.g., model-specific aspect ratios, duration limits, and audio flags), causing un-enriched LLM judges to grant false passes (TC = 1.000) to production-breaking bugs.
3. **The Capability-Matrix Enrichment Fix:** We architect and validate a machine-readable capability matrix enrichment layer that bridges static JSON schemas and backend runtime registries, restoring robust discrimination gaps (≥ 0.191) across all evaluated servers and expanding the image generation discrimination margin by +36.1%.
4. **A Three-Layer Generative Media Evaluation Taxonomy:** We formalize a decoupled architectural framework separating Infrastructure Liveness (Layer 0), Orchestration Correctness (Layer 1), and Perceptual Media Quality (Layer 2), enabling sub-second, zero-cost CI pull request gating.

2 Background, Related Work & Positioning

2.1 Agent Tool-Calling Benchmarks & Synthetic Generation

Agent evaluation has progressed from single-function prediction (e.g., ToolBench, Berkeley Function Calling Leaderboard / BFCL) to multi-turn, policy-grounded execution benchmarks. However, the majority of existing benchmarks require live tool execution (APIGen, TOUCAN) or hand-curated simulated environments (τ-bench, ToolSandbox).

Spec-only synthetic generation (such as DiGiT-TC and FuncBenchGen) has emerged to generate evaluation data directly from signatures. Agent Seer advances this paradigm by synthesizing not only user scenarios and oracle tool calls, but also calibrated, schema-valid mock tool responses that enable multi-turn conversational expansion without runtime dependencies.

2.2 The Agent Seer Formulation

Agent Seer converts raw MCP tool specifications into self-contained evaluation harnesses through a four-stage pipeline: Tool Interpretation (Stage 1), Scenario Generation (Stage 2), Mock Output Generation (Stage 3), and Multi-Turn Expansion (Stage 4). Transcripts are evaluated using an unsupervised LLM-as-judge rubric measuring **Tool-Calling Correctness (TC)** across 14 decomposed sub-dimensions and **Conversational Coherence (Coh)** across 5 qualitative dimensions.

2.3 Generative-Media MCPs vs. Conventional Tool Suites

The original Agent Seer study evaluated seven open-source MCP specifications: Redis, Git, Filesystem, Elasticsearch, Slack, Selenium, and Illustrator. These tools primarily represent deterministic data stores, operating system utilities, or UI automation frameworks where input parameters map directly to explicit JSON Schema properties.

In contrast, generative-media MCP suites introduce fundamental architectural differences:

- **Multi-Model Endpoints:** A single tool (e.g., `nanobanana_image_generation` or `veo_t2v`) routes requests across disparate foundation models (e.g., Gemini 2.5 Flash, Gemini 3 Pro, Veo 2.0, Veo 3.1) with non-overlapping feature sets.
- **Hidden Capability Constraints:** Valid parameter combinations (such as whether `image_size: "4K"` or `generate_audio: true` is permitted) are governed by internal model registry structs rather than JSON Schema enums.
- **High Execution Costs:** Live evaluation costs $1.00–$5.00+ per run with 60–120s latencies, making execution-free synthetic evaluation essential.

These characteristics make generative-media MCP servers an ideal stress-test for specification-driven evaluation, while exposing the critical limitation of schema-blind judging.

3 Specification-Driven Evaluation Pipeline & Scoring Mechanics

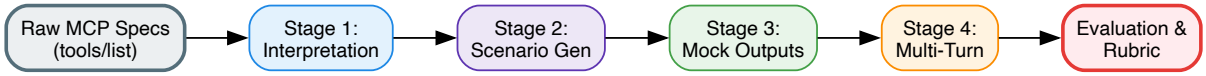

Figure 1: The Four-Stage Agent Seer Specification-Driven Evaluation Pipeline. Converts raw JSON-RPC schemas into complete evaluation trajectories using structured, unsupervised LLM tasks.

3.1 Four-Stage Spec-Driven Pipeline Deconstruction

The synthetic evaluation pipeline transforms raw MCP `tools/list` declarations into validated evaluation harnesses through four sequential stages:

Stage 1: Tool Interpretation (Semantic Feature Extraction)

Raw tool definitions are expanded into a 5-dimensional semantic representation: `tool_name`, `what_it_does`, `what_it_needs` (required vs. optional parameters and type constraints), `why_its_used`, and `enterprise_context` classification tags.

Stage 2: Scenario Generation (Simple vs. Complex & Oracle Workflows)

Using Stage 1 semantic summaries, the generator produces task scenarios across two complexity tiers (Simple single-intent tasks and Complex multi-tool enterprise workflows). To eliminate generator selection bias, prompts inject a strict coverage guarantee:

$$\forall t \in \mathcal{T}, \qquad \exists s \in \mathcal{S} \qquad \text{such that} \qquad t \in \text{Workflow}(s) \tag{1}$$

Where $\mathcal{T}$ is the set of N available tools in the MCP suite and $\mathcal{S}$ is the set of generated scenarios. On `mcp-veo-go` ($N = 6$), Stage 2 yielded 15 scenarios with **100% tool coverage (0 uncovered tools)**. Every scenario outputs an `agent_workflow` containing expected tool names, fully bound arguments, and step explanations as a held-out oracle.

Stage 3: Mock Output Generation with Grounding Tiers

Stage 3 synthesizes realistic tool response payloads for each step in the oracle workflow. Mocks are categorized into three grounding tiers:

- **High Grounding:** Grounded in verified runtime execution schemas or real success response fixtures (`"confidence": "high"`).
- **Medium Grounding:** Grounded in analogous tool outputs within the same server family (`"confidence": "medium"`).
- **Low Grounding:** Pure ungrounded LLM synthesis (`"confidence": "low"`).

In our reproduction, seeding Stage 3 with real response fixtures (`spike/seed_outputs.json`) achieved **84.2% High Grounding (16/19 steps)** and **15.8% Medium Grounding (3/19 steps)**, with **0.0% Low Grounding hallucinations**.

Stage 4: Multi-Turn Expansion

Composite workflows are segmented at natural phase boundaries to create multi-turn dialogues exercising multi-step dependency chains and multi-hop information synthesis (aligned with BFCL v3 patterns), where follow-up prompts explicitly reference identifiers emitted in prior mock outputs.

3.2 Decomposed LLM-as-Judge Rubric

Evaluation separates assessment into **Tool-Calling Correctness (TC)** and **Conversational Coherence (Coh)**.

Idx	Category	Subdimension (k)	Scope	Evaluation Focus
1	**Usage**	`necessity`	Always	Was a tool call required, or could the LLM answer directly?
2		`overuse_detection`	Diagnostic	Did the agent make redundant or unprompted calls? (Diagnostic; excluded from aggregate)
3	**Selection**	`correctness`	Always	Does the tool choice match the requested functional intent?
4		`specificity`	Always	Was the most specialized tool selected over generic tools?
5		`completeness`	Always	Were all necessary tools selected to satisfy the task?
6	**Arguments**	`completeness`	Always	Are all mandatory schema parameters provided?
7		`name_accuracy`	Always	Do parameter keys match the schema exactly (case-sensitive)?
8		`value_accuracy`	Always	Are values grounded, valid, and aligned with prompt/context?
9		`type_compliance`	Always	Do values match expected types (string, int, array, object)?
10		`format_compliance`	Always	Do values follow formats (URI schemes, enums, bounds)?
11		`relevancy`	Always	Are arguments free of ungrounded or extraneous keys?
12	**Ordering**	`sequence_logic`	Tools > 1	Is execution order logical across dependent steps?
13		`dependency_handling`	Tools > 1	Are output values from earlier steps piped correctly?
14		`execution_efficiency`	Tools > 1	Is the execution path optimal without redundant hops?

Note: Subdimension scores $x_k \in [0, 10]$ normalize via $\text{norm}_{10}(x_k) = \max\big(0.0, \min\big(1.0, \frac{x_k}{10.0}\big)\big)$. Aggregate dimension formulas $D_{\text{usage}}, D_{\text{selection}}, D_{\text{arguments}}, D_{\text{ordering}}$ are detailed below.

Dimension and composite scores aggregate as:

1. $D_{\text{usage}} = \text{norm}_{10}\big(x_{\text{necessity}}\big)$
2. $D_{\text{selection}} = \frac{1}{3}\big(\text{norm}_{10}(x_{\text{cor}}) + \text{norm}_{10}\big(x_{\text{spec}}\big) + \text{norm}_{10}\big(x_{\text{comp}}\big)\big)$
3. $D_{\text{arguments}} = \frac{1}{6}\sum_{k \in \mathcal{K}_{\text{arg}}} \text{norm}_{10}(x_k)$
4. $D_{\text{ordering}} = \frac{1}{\lvert\mathcal{K}_{\text{ord}}\rvert}\sum_{k \in \mathcal{K}_{\text{ord}}} \text{norm}_{10}(x_k)$ (omitted when $M = 1$)
5. $\text{TC}_{\text{overall}} = \frac{1}{\lvert\mathcal{D}_{\text{active}}\rvert}\sum_{d \in \mathcal{D}_{\text{active}}} D_d$

3.3 Cascading Penalty Mechanics & Failure Propagation Proof

Naive LLM judges suffer from **linear averaging dilution**: if an agent emits a tool call with a completely invalid parameter name, a linear average over 6 argument subdimensions yields $D_{\text{arguments}} = \frac{5}{6} = 0.833$ and an inflated composite TC $= 0.944$ (a False Pass).

To prevent dilution, the rubric enforces **mandatory cascading penalties**:

- **Case 1 (Invalid Name or Missing Required):** If `name_accuracy` ≤ 2 or `completeness` ≤ 2, the judge forces `value_accuracy`, `type_compliance`, and `format_compliance` to ≤ 2, collapsing $D_{\text{arguments}} \leq 0.333$ and TC ≤ 0.778.
- **Case 2 (Invalid Parameter Value):** If `value_accuracy` ≤ 3, the cascade forces `type_compliance`, `format_compliance`, and `relevancy` to ≤ 3, collapsing $D_{\text{arguments}} \leq 0.467$ and TC ≤ 0.800.

Mathematical Proof: When a critical parameter name error occurs (e.g., passing `ratio` instead of `aspect_ratio` in case `A6`):
1. `name_accuracy` drops to 0.0.
2. The cascade forces `value_accuracy` ≤ 0.0, `type_compliance` ≤ 0.0, and `format_compliance` ≤ 0.0.
3. `completeness` drops to 0.0 because the required parameter was omitted.
4. $D_{\text{arguments}} = 0.000$, and composite TC $= \frac{1.0+1.0+0.0}{3} = 0.667$.

A single syntax error immediately eliminates 33.3% of the total available score.

3.4 Conversational Coherence (Coh) Formulation

Conversational Coherence evaluates natural language output across 5 dimensions (Logical Flow, Completeness, Conciseness, Relevance, Context Retention) on a 3-point scale ($1 = \text{Poor}, 2 = \text{Adequate}, 3 = \text{Good}$). Scores normalize via $\text{norm}_3(x) = \frac{x-1}{2.0}$ and aggregate via arithmetic mean:

$$\text{Coh}_{\text{overall}} = \frac{1}{|\mathcal{V}_{\text{active}}|} \sum_{v \in \mathcal{V}_{\text{active}}} \text{norm}_3(v) \tag{2}$$

4 Primary Finding: Schema-Blindness & Capability-Matrix Grounding

4.1 The Mechanism of Schema-Blindness

In modern MCP server implementations, tool schemas published via `tools/list` are decoupled from backend runtime registries:
1. **Loose Schema Typing:** Parameter schemas declare permissive generic types (e.g., `aspect_ratio: { "type": "string" }`).
2. **Hidden Backend Constraints:** Exact compatibility rules reside in Go model registries (`SupportedVeoModels`, `capabilities.json`).
3. **Judge Information Asymmetry:** The LLM judge evaluates calls strictly against the schema in its prompt. If a constraint is absent from `tools/list`, the judge has zero basis to penalize the violation.

4.2 Empirical Baseline False Passes

During baseline runs with Gemini 2.5 Flash (Temperature 0.0):
- **Veo Case `A1-wrong-model-value`:** The agent called `veo_t2v` with `model: "veo-2.0-generate-001"` and `generate_audio: true`. Veo 2.0 physically rejects audio generation. Because `tools/list` did not document model-specific audio compatibility, the baseline judge awarded a **flawless TC = 1.000**.
- **Nanobanana Case `NB1-illegal-size-on-2.5`:** The agent called `gemini-2.5-flash-image` with `image_size: "4K"`. Flash 2.5 does not support resolution scaling. The un-enriched judge granted a near-pass score of **TC = 0.944**.

4.3 Capability Matrix Injection Architecture

To resolve schema-blindness, we architected a **Capability Matrix Enrichment** layer that extracts runtime registries and appends a machine-readable capability contract into the judge's prompt context:

```
// Enriched Prompt Injection: CRITICAL BACKEND MODEL CAPABILITY MATRIX (MUST ENFORCE)
{
  "veo-2.0-generate-001": {
    "SupportsGenerateAudio": "false",
    "SupportedAspectRatios": ["16:9"],
    "SupportedDurations": [5],
    "MaxVideos": 4,
    "SupportsFirstLast": false,
    "SupportsReferenceImage": false
  },
  "veo-3.1-generate-001": {
    "SupportsGenerateAudio": "true",
    "SupportedAspectRatios": ["16:9"],
    "SupportedDurations": [4],
    "MaxVideos": 4,
    "SupportsFirstLast": true,
    "SupportsReferenceImage": false
  }
}
```

4.4 Restoration of Clean Discrimination

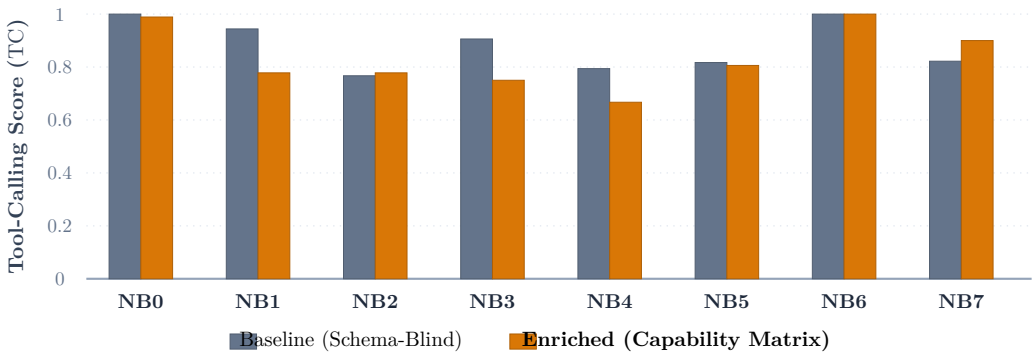

Figure 2: Tool-Calling Correctness (TC) scores on the Nanobanana server. Capability Matrix enrichment resolves the schema-blindness vulnerability, collapsing false passes on faulty runs (NB1, NB3, NB4) while preserving correct baseline results (NB0, NB6).

Upon injecting the capability matrix, the judge immediately enforced hidden constraints:

- **Veo Case A1:** TC collapsed from **1.000 → 0.800** ($\Delta = -0.200$). Cascading penalties collapsed argument subscores to 0.2, correctly logging failure taxonomy flags `['argument_value', 'argument_type', 'argument_format', 'argument_relevancy']`.
- **Nanobanana Case NB1:** TC collapsed from **0.944 → 0.778** ($\Delta = -0.166$). Argument score collapsed to 0.333.
- **Valid Calls Preserved:** Correct baseline cases remained pristine (`A0-correct`: 0.994, `NB0-correct`: 0.989, `NB6-correct`: 1.000, `LY0-correct`: 1.000).

5 Experimental Evaluation across Three Generative-Media MCP Server Suites

All empirical evaluations were executed using Gemini 2.5 Flash (Primary Judge, Temperature 0.0) and cross-validated with Gemini 2.5 Pro and Gemma 2 27B IT, evaluated across 26 distinct test cases in the reproduction repository (`spike/artifacts/`).

5.1 Comprehensive Multi-Server Summary

Server Suite	Evaluation Run	Mean Correct TC	Mean Broken TC	Discrimination Gap	Taxonomy Hits
Veo (Video)	Baseline	1.000	0.768	0.232	7/9 (77.8%)
Veo (Video)	Enriched	0.994	0.796	0.198	9/9 (100.0%)
Nanobanana (Image)	Baseline	1.000	0.842	0.158	6/6 (100.0%)
Nanobanana (Image)	Enriched	0.994	0.780	0.215 (+36.1% gap)	5/6 (83.3%)
Lyria (Music)	Baseline	1.000	0.752	0.248	5/5 (100.0%)
Lyria (Music)	Enriched	1.000	0.809	0.191	5/5 (100.0%)

5.2 Server Suite 1: Google Veo (`mcp-veo-go`)

The Veo suite evaluates 11 hand-authored transcripts covering 6 distinct tools and 9 injected failure modes.

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline (Flash)	Baseline (Pro)	Enriched (Flash)
`A0-correct`	Correct	Text-to-video (16:9, audio, valid GCS bucket)	None	**1.000**	1.000	**0.994**
`A1-wrong-model-value`	Broken	Veo 2.0 requesting `generate_audio=true`	`argument_val`	**1.000** False	0.839	**0.800**
`A2-illegal-enum`	Broken	Veo 3.1 with `aspect_ratio: "21:9"`	`argument_fmt`	**0.956** Near	0.867	**0.789**
`A3-hallucinated-model`	Broken	Model ID `veo-3.5-ultra` (not in spec)	`argument_val`	**0.822**	0.822	**0.800**
`A4-missing-required`	Broken	Omitted required parameter `prompt`	`arg_comp`	**0.722**	0.778	N/A
`A5-wrong-tool`	Broken	Invoked `veo_i2v` with no image provided	`selection`	**0.444**	0.494	N/A
`A6-wrong-param-names`	Broken	Parameters `ratio` & `gcs_bucket` passed	`argument_name`	**0.667**	0.794	N/A
`B0-correct`	Correct	Image-to-video with valid GCS source	None	**1.000**	1.000	N/A
`B1-wrong-tool`	Broken	Invoked `veo_t2v` ignoring provided image	`selection`	**0.656**	0.500	N/A
`B2-missing-req-img`	Broken	Omitted required `image_uri` in `veo_i2v`	`arg_comp`	**0.778**	0.794	N/A
`B3-malformed-uri`	Broken	Passed local path `in.png` instead of `gs://`	`argument_fmt`	**0.867**	0.911	N/A

5.3 Server Suite 2: Gemini Image / Nanobanana (`mcp-nanobanana-go`)

The Nanobanana suite evaluates 8 test cases covering multimodal inputs, resolution scaling, aspect ratio limits, and parameter naming rules.

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline TC	Enriched TC
`NB0-correct`	Correct	Text-to-image (Gemini 3.1 Flash, 16:9, 2K)	None	**1.000**	**0.989**
`NB1-illegal-size-on-2.5`	Broken	`gemini-2.5-flash-image` with `image_size: "4K"`	`argument_val`	**0.944** Near	**0.778**
`NB2-illegal-aspect-ratio`	Broken	Flash 2.5 with ultra-tall aspect ratio `1:8`	`argument_fmt`	**0.767**	**0.778**
`NB3-hallucinated-model`	Broken	Hallucinated `imagen-3.5-ultra-banana`	`argument_val`	**0.906** Near	**0.750**

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline TC	Enriched TC
`NB4-missing-req-prompt`	Broken	Omitted required `prompt` parameter	`arg_comp`	**0.794**	**0.667**
`NB5-wrong-param-names`	Broken	Invalid names `ratio` & `bucket`	`argument_name`	**0.817**	**0.806**
`NB6-correct-image-to-image`	Correct	Gemini 3 Pro with valid input image array	None	**1.000**	**1.000**
`NB7-malformed-images-type`	Broken	`images` passed as bare string (not array)	`argument_type`	**0.822**	**0.900**

Key Metric: Capability matrix enrichment expanded Nanobanana's discrimination gap from **0.158 to 0.215**, representing a **+36.1% expansion in discriminating power**.

5.4 Server Suite 3: Google Lyria (`mcp-lyria-go`)

The Lyria suite evaluates 7 test cases covering audio generation durations, parameter name variations (`model_id` vs `model`), GCS bucket naming, and negative prompt conditioning.

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline TC	Enriched TC
`LY0-correct`	Correct	Lyria 3 Clip (30s lofi jazz, GCS output)	None	**1.000**	**1.000**
`LY1-wrong-model-param-name`	Broken	Passed `model` instead of required `model_id`	`argument_name`	**0.739**	**0.778**
`LY2-wrong-bucket-param-name`	Broken	Passed `gcs_bucket_uri` instead of expected	`argument_name`	**0.667**	**0.889**
`LY3-hallucinated-model`	Broken	Hallucinated `lyria-ultra-composer-001`	`argument_val`	**0.800**	**0.806**
`LY4-missing-required-prompt`	Broken	Omitted required `prompt` parameter	`arg_comp`	**0.778**	**0.794**
`LY5-correct-full-track`	Correct	Lyria 3 Pro (150s full orchestral score)	None	**1.000**	**1.000**
`LY6-malformed-sample-count`	Broken	`sample_count: -5` (violates minimum constraint)	`argument_fmt`	**0.778**	**0.778**

5.5 Cross-Server Production Pipeline Evaluation

We also validated multi-turn capabilities over a four-step cross-server media pipeline (`cross_server_media_production`) where inputs are chained from Lyria to Nanobanana to Veo.

Scenario	Injected Fault	Total TC	Selection	Arguments	Ordering
CS0-correct-pipeline	None (Baseline)	**0.912**	1.000	0.717	0.933
CS1-broken-uri-pipe	Broken URI (Step 2)	**0.817**	1.000	0.367	0.900
CS2-aspect-ratio-mismatch	Aspect Ratio Mismatch	**0.792**	1.000	0.500	0.667
CS3-broken-pipeline-ordering	Out-of-order execution	**0.517**	0.733	0.333	0.000

The cross-server run confirms that while selection remains resilient, out-of-order execution (**CS3**) completely derails the model's pipeline state representation, collapsing the Ordering dimension to **0.000** and overall TC to **0.517**.

6 Discussion: Architectural Boundaries & Evaluation Robustness

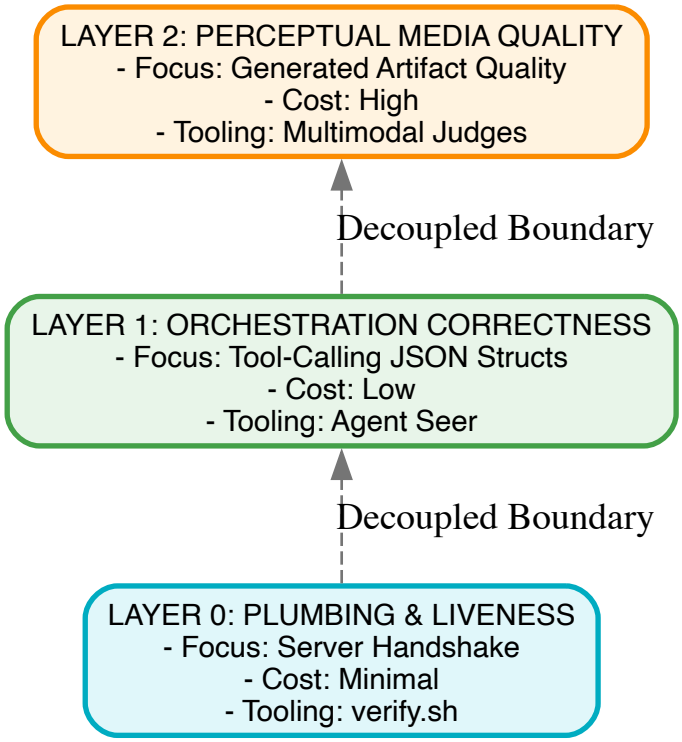

Figure 3: The Three-Layer Generative Media Evaluation Stack. Agent Seer isolates and evaluates Layer 1 (Orchestration Correctness) independently of Layer 0 (Infrastructure) and Layer 2 (Perceptual Media Quality) to minimize latency, costs, and defect attribution noise.

6.1 Why Orchestration Must Be Decoupled from Perceptual Evaluation

1. **Defect Attribution Precision:** In an integrated end-to-end test, a video generation failure could stem from an orchestration error (agent passed an incompatible aspect ratio), a network error (GCS bucket permission denial), or a diffusion failure (model generated visual artifacts). Layer 1 evaluation isolates agent cognitive defects with zero confounding noise from downstream generative models.
2. **Cost and Latency Decoupling:** Rendering 100 scenario permutations through live video diffusion models on Vertex AI takes hours and incurs substantial GPU billing. Agent Seer evaluates the exact same 100 scenario trajectories in seconds at near-zero inference cost using synthetic mock outputs.
3. **Deterministic CI Gating:** Layer 1 transcript evaluation at Temperature 0.0 with explicit capability contracts provides a deterministic, reproducible gate for continuous integration pull requests.

6.2 Judge Circularity Mitigations & Out-of-Family Robustness

Evaluating LLM outputs using another LLM introduces risks of self-evaluation bias, family circularity, and scoring drift. The Agent Seer architecture deploys four defensive mitigations:

- **Evaluator-Generator Capacity Asymmetry:** The scenario generation pipeline utilizes `gemini-2.5-flash-lite` operating at Temperature 0.7 with structured JSON constraints to encourage creative scenario diversity. Conversely, the evaluation harness utilizes `gemini-2.5-flash` or `gemini-2.5-pro` strictly pinned at **Temperature 0.0** to guarantee determinism.
- **Out-of-Family Replication Dynamics:** In the published paper (§5), Karumuri et al. re-scored all 391 evaluation records using Alibaba's `Qwen3.5-122B`, demonstrating a paired Pearson correlation of $r \approx 0.79$ on Tool-Calling Correctness and a Spearman rank correlation of $\rho = 0.86$ across MCP server rankings. Our reproduction suite implements `spike/gemma_client.py` for out-of-family judging via Vertex AI Model Garden hosting **Gemma 2 27B IT**.

- **Prompt & Task Decoupling:** The TC Judge and Coherence Judge execute in completely isolated contexts with zero shared memory.
- **Taxonomy-Constrained Rubric:** Discrete categorical fault attribution is forced rather than unconstrained floating-point scoring.

7 Conclusion

This work presented an empirical reproduction and extension of the specification-driven evaluation methodology of **Agent Seer** across three production-grade generative-media MCP suites. While the methodology successfully eliminates the cold-start benchmark curation bottleneck and achieves 100% tool coverage, our empirical findings reveal that raw JSON schemas are vulnerable to **Schema-Blindness**. Injecting machine-readable capability matrices restores clean discrimination gaps (≥ 0.191) and expands discrimination margins by $+36.1\%$, establishing a robust, decoupled foundation for continuous integration evaluation of autonomous AI agents.

APPENDIX A Independent Reproduction & Verification Guide

To independently reproduce the empirical scores, deltas, and tables documented in this technical report, execute the following commands in the workspace:

```
# 1. Run Baseline Veo Discrimination Test (Flash & Pro Judges)
python3 agent-seer-mcp-tool-calling/spike/discrimination_test.py --second-judge

# 2. Run Nanobanana Baseline & Enriched Discrimination Tests
python3 agent-seer-mcp-tool-calling/spike/runner.py --server nanobanana
python3 agent-seer-mcp-tool-calling/spike/runner.py --server nanobanana --enriched

# 3. Run Lyria Baseline & Enriched Discrimination Tests
python3 agent-seer-mcp-tool-calling/spike/runner.py --server lyria
python3 agent-seer-mcp-tool-calling/spike/runner.py --server lyria --enriched

# 4. Verify Mathematical Scoring Assertions
python3 -c '
import sys
sys.path.insert(0, "agent-seer-mcp-tool-calling/spike")
import scoring

# Verify Perfect Call Score
perfect = {
    "usage": {"necessity": 10, "overuse_detection": 10},
    "selection": {"correctness": 10, "specificity": 10, "completeness": 10},
    "ordering": {"not_applicable": True},
    "arguments": {"completeness": 10, "name_accuracy": 10, "value_accuracy": 10,
"type_compliance": 10, "format_compliance": 10, "relevancy": 10},
    "failures": [], "rationale": "perfect"
}
assert scoring.aggregate_tc(perfect)["tc_overall"] == 1.0

# Verify Cascading Parameter Name Collapse (1.0 -> 0.667)
cascaded_name = {
    "usage": {"necessity": 10, "overuse_detection": 10},
    "selection": {"correctness": 10, "specificity": 10, "completeness": 10},
    "ordering": {"not_applicable": True},
    "arguments": {"completeness": 0, "name_accuracy": 0, "value_accuracy": 0,
"type_compliance": 0, "format_compliance": 0, "relevancy": 0},
    "failures": ["argument_name"], "rationale": "bad name"
}
assert round(scoring.aggregate_tc(cascaded_name)["tc_overall"], 3) == 0.667
print("All mathematical scoring assertions verified successfully.")
'
```


```
# 1. Run Baseline Veo Discrimination Test (Flash & Pro Judges)
python3 agent-seer-mcp-tool-calling/spike/discrimination_test.py --second-judge

# 2. Run Nanobanana Baseline & Enriched Discrimination Tests
python3 agent-seer-mcp-tool-calling/spike/runner.py --server nanobanana
python3 agent-seer-mcp-tool-calling/spike/runner.py --server nanobanana --enriched

# 3. Run Lyria Baseline & Enriched Discrimination Tests
python3 agent-seer-mcp-tool-calling/spike/runner.py --server lyria
python3 agent-seer-mcp-tool-calling/spike/runner.py --server lyria --enriched

# 4. Verify Mathematical Scoring Assertions
```

```
python3 -c '
import sys
sys.path.insert(0, "agent-seer-mcp-tool-calling/spike")
import scoring

# Verify Perfect Call Score
perfect = {
    "usage": {"necessity": 10, "overuse_detection": 10},
    "selection": {"correctness": 10, "specificity": 10, "completeness": 10},
    "ordering": {"not_applicable": True},
    "arguments": {"completeness": 10, "name_accuracy": 10, "value_accuracy": 10,
"type_compliance": 10, "format_compliance": 10, "relevancy": 10},
    "failures": [], "rationale": "perfect"
}
assert scoring.aggregate_tc(perfect)["tc_overall"] == 1.0

# Verify Cascading Parameter Name Collapse (1.0 -> 0.667)
cascaded_name = {
    "usage": {"necessity": 10, "overuse_detection": 10},
    "selection": {"correctness": 10, "specificity": 10, "completeness": 10},
    "ordering": {"not_applicable": True},
    "arguments": {"completeness": 0, "name_accuracy": 0, "value_accuracy": 0,
"type_compliance": 0, "format_compliance": 0, "relevancy": 0},
    "failures": ["argument_name"], "rationale": "bad name"
}
assert round(scoring.aggregate_tc(cascaded_name)["tc_overall"], 3) == 0.667
print("All mathematical scoring assertions verified successfully.")
'
```